

Report

Group name: P & Team

GitHub link: [GitHub P & Team](#)

video link: [Video](#)

1. Shard Key Chosen and Justification

The shard key selected for the `MenuItem` table is `item_id`.

This choice is justified based on the following key properties:

- **High Cardinality:**
`item_id` is the primary key and uniquely identifies each row. This ensures maximum cardinality, allowing the hash function to distribute records evenly across shards without clustering.
- **Query Alignment:**
All API endpoints use `item_id` in their `WHERE` clause (e.g., GET, PUT, DELETE operations). This enables direct routing to the correct shard without querying all shards.
- **Stability:**
`item_id` is immutable after insertion. Since shard keys must not change, this prevents costly data movement between shards.

Thus, `item_id` satisfies all critical properties required for an efficient shard key.

Local MySQL - Warning - not supported

Administration Schemas Query 1 Dump20260402*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

Object Info Session

No object selected

Result Grid

Filter Rows: Search Export:

Limit to 1000 rows

```
1 USE mydb;
2 SELECT COUNT(*) FROM MenuItem;
3 SELECT
4 (SELECT COUNT(*) FROM shard_0_menuitem) +
5 (SELECT COUNT(*) FROM shard_1_menuitem) +
6 (SELECT COUNT(*) FROM shard_2_menuitem);
```

Result 4

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
✓	56	22:41:27	SELECT COUNT(*) FROM MenuItem LIMIT 0, 1000	1 row(s) returned 0.00054 sec / 0.000...

Query Completed

Local MySQL - Warning - not supported

Administration Schemas Query 1 Dump20260402*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

Object Info Session

No object selected

Result Grid

Filter Rows: Search Export:

Limit to 1000 rows

```
1 USE mydb;
2 SELECT COUNT(*) FROM MenuItem;
3 SELECT
4 (SELECT COUNT(*) FROM shard_0_menuitem) +
5 (SELECT COUNT(*) FROM shard_1_menuitem) +
6 (SELECT COUNT(*) FROM shard_2_menuitem);
```

Result 5

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
✓	57	22:41:59	SELECT (SELECT COUNT(*) FROM shard_0_menuitem) + (SELECT COUNT(*) FROM shard_1_menuitem) + (SELECT COU...	1 row(s) returned 0.00064 sec / 0.000...

Query Completed

Local MySQL - Warning - not supported

Administration Schemas Query 1 Dump20260402*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

Object Info Session

No object selected

Result Grid

Filter Rows: Search Export:

19

Result 5

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
57	22:41:59	SELECT (SELECT COUNT(*) FROM shard_0_menuitem) + (SELECT COUNT(*) FROM shard_1_menuitem) + (SELECT COU...	1 row(s) returned	0.00064 sec / 0.000...

Query Completed

Local MySQL - Warning - not supported

Administration Schemas Query 1 Dump20260402*

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

Object Info Session

No object selected

Result Grid

Filter Rows: Search Export:

18

19

20

21

22

23

Result 8

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
60	22:44:21	SELECT 'shard_0', COUNT(*) FROM shard_0_menuitem UNION SELECT 'shard_1', COUNT(*) FROM shard_1_menuitem UNI...	3 row(s) returned	0.0012 sec / 0.00000...

Query Completed

2. Partitioning Strategy Used and Why

A **hash-based partitioning strategy** was used:

`shard_id = item_id % 3`

Why Hash-Based Partitioning?

- Ensures **uniform distribution** of data across shards
- Works effectively with sequential IDs (like 701–720)
- Avoids **hotspots** seen in range partitioning
- Requires **no lookup table**, unlike directory-based partitioning
- Provides **O(1) routing** with simple modulo computation

Compared to alternatives:

- Range-based → causes uneven load with sequential inserts
- Directory-based → introduces lookup overhead and complexity

Hence, hash-based partitioning is the most efficient and scalable choice.

3. Query Routing Implementation

Query routing is implemented using a **deterministic shard router**.

Routing Logic

Shard is computed using:

`shard = item_id % 3`

-
- Routing functions:
 - `get_shard_connection(item_id)` → returns correct shard connection
 - `get_all_shard_connections()` → used for multi-shard queries

Behavior of Different Queries

- **Single Item Queries (GET, PUT, DELETE):**
 - Routed to exactly one shard
 - No scatter-gather required
- **Insert (POST):**
 - Inserted directly into the correct shard
- **Full Table Query (GET /menu):**
 - Queries all shards
 - Results merged and sorted
- **Range Query:**
 - Queries all shards
 - Merges filtered results

Evidence from Implementation

The routing logic is clearly reflected in the backend routes:

- Single item query:
 - routed using `get_shard_connection(item_id)`
- Multi-shard query:
 - uses `get_all_shard_connections()`

This confirms efficient routing without unnecessary shard access.

THUNDER CLIENT

project (Workspace)

GET http://127.0.0.1:5001/api/menu

Status: 200 OK Size: 3.26 KB Time: 214 ms

Query Parameters

Response

```
{
  "data": [
    {
      "availability": 1,
      "category_id": 601,
      "item_id": 701,
      "item_name": "Chicken Biryani",
      "price": "230.00",
      "restaurant_id": 401
    },
    {
      "availability": 1,
      "category_id": 601,
      "item_id": 702,
      "item_name": "Veg Biryani",
      "price": "180.00",
      "restaurant_id": 401
    },
    {
      "availability": 1,
      "category_id": 601,
      "item_id": 703,
      "item_name": "Mutton Biryani",
      "price": "250.00",
      "restaurant_id": 401
    }
  ]
}
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
✓ B+ Tree synced from shard 2 (port 3309) (7 items)
✓ WAL clean - no recovery needed
✓ B+ Tree fully synced from all shards
  * Serving Flask app 'app'
  * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
  * Running on http://127.0.0.1:5001
  Press CTRL+C to quit
  * Restarting with stat
✓ B+ Tree synced from shard 0 (port 3307) (7 items)
✓ B+ Tree synced from shard 1 (port 3308) (5 items)
✓ B+ Tree synced from shard 2 (port 3309) (7 items)
✓ WAL clean - no recovery needed
✓ B+ Tree fully synced from all shards
  * Debugger is active!
  * Debugger PIN: 761-059-450
127.0.0.1 - - [18/Apr/2026 05:35:02] "GET /api/menu HTTP/1.1" 200 -
```

THUNDER CLIENT

project (Workspace)

GET http://127.0.0.1:5001/api/menu/701

Status: 200 OK Size: 213 Bytes Time: 84 ms

Query Parameters

Response

```
{
  "data": {
    "availability": 1,
    "category_id": 601,
    "item_id": 701,
    "item_name": "Chicken Biryani",
    "price": "230.00",
    "restaurant_id": 401
  },
  "shard_used": 2,
  "status": "success"
}
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
✓ WAL clean - no recovery needed
✓ B+ Tree fully synced from all shards
  * Serving Flask app 'app'
  * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
  * Running on http://127.0.0.1:5001
  Press CTRL+C to quit
  * Restarting with stat
✓ B+ Tree synced from shard 0 (port 3307) (7 items)
✓ B+ Tree synced from shard 1 (port 3308) (5 items)
✓ B+ Tree synced from shard 2 (port 3309) (7 items)
✓ WAL clean - no recovery needed
✓ B+ Tree fully synced from all shards
  * Debugger is active!
  * Debugger PIN: 761-059-450
127.0.0.1 - - [18/Apr/2026 05:35:02] "GET /api/menu HTTP/1.1" 200 -
127.0.0.1 - - [18/Apr/2026 05:37:24] "GET /api/menu/701 HTTP/1.1" 200 -
```

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET 127.0.0.1:5001/api/menu/range?start=... just now

GET 127.0.0.1:5001/api/menu/701 just now

GET 127.0.0.1:5001/api/menu just now

Query

Headers 2 Auth Body Tests Pre Run

Query Parameters

start	701
end	710
parameter	value

Status: 200 OK Size: 1.84 KB Time: 172 ms

Response

Headers 5 Cookies Results Docs

```
2 "count": 9,
3 "data": [
4   {
5     "availability": 1,
6     "category_id": 601,
7     "item_id": 701,
8     "item_name": "Chicken Biryani",
9     "price": "230.00",
10    "restaurant_id": 401,
11    "shard_source": 2
12  },
13  {
14    "availability": 1,
15    "category_id": 601,
16    "item_id": 702,
17    "item_name": "Veg Biryani",
18    "price": "180.00",
19    "restaurant_id": 401,
20    "shard_source": 0
21  },
22  {
23    "availability": 1,
24    "category_id": 602,
25    "item_id": 703,
26    "item_name": "Margherita Pizza",
27    "price": "299.00",
28    "restaurant_id": 402,
29    "shard_source": 1
30  },
31  ]
32 }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
127.0.0.1 -- [18/Apr/2026 05:37:24] "GET /api/menu/701 HTTP/1.1" 200 -
127.0.0.1 -- [18/Apr/2026 05:38:26] "GET /api/menu/702 HTTP/1.1" 200 -
127.0.0.1 -- [18/Apr/2026 05:38:50] "GET /api/menu/703 HTTP/1.1" 200 -
127.0.0.1 -- [18/Apr/2026 05:39:20] "GET /api/menu/range?start=701&end=710 HTTP/1.1" 200 -
```

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST 127.0.0.1:5001/api/menu just now

GET 127.0.0.1:5001/api/menu/range?start=... just now

GET 127.0.0.1:5001/api/menu/701 just now

GET 127.0.0.1:5001/api/menu just now

POST

Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "item_id": 721,
3   "item_name": "Test Item",
4   "price": 99.99,
5   "restaurant_id": 401,
6   "category_id": 601,
7   "availability": 1
8 }
```

Status: 201 CREATED Size: 102 Bytes Time: 74 ms

Response

Headers 5 Cookies Results Docs

```
1 {
2   "message": "Successfully added Test Item to shard 1!",
3   "shard_used": 1,
4   "status": "success"
5 }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
127.0.0.1 -- [18/Apr/2026 05:40:43] "GET /api/menu HTTP/1.1" 200 -
[DEBUG] WAL: Logged INSERT - {'item_id': 721}
127.0.0.1 -- [18/Apr/2026 05:41:16] "POST /api/menu HTTP/1.1" 201 -
```

THUNDER CLIENT

project (Workspace)

New Request

Activity Collections Env

filter activity

PUT 127.0.0.1:5001/api/menu/721 just now

POST 127.0.0.1:5001/api/menu just now

GET 127.0.0.1:5001/api/menu/range?start=... just now

GET 127.0.0.1:5001/api/menu/701 just now

GET 127.0.0.1:5001/api/menu just now

PUT http://127.0.0.1:5001/api/menu/721 Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "price": 149.99,
3   "availability": 1
4 }
5
```

Status: 200 OK Size: 103 Bytes Time: 96 ms

Response Headers Cookies Results Docs

```
1 {
2   "message": "Successfully updated item 721 on shard 1!",
3   "shard_used": 1,
4   "status": "success"
5 }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
127.0.0.1 -- [18/Apr/2026 05:41:16] "POST /api/menu HTTP/1.1" 201 -
WAL: Logged UPDATE - {'item_id': 721, 'price': 149.99, 'availability': 1}
WAL: Committed UPDATE - {'item_id': 721, 'price': 149.99, 'availability': 1}
127.0.0.1 -- [18/Apr/2026 05:42:20] "PUT /api/menu/721 HTTP/1.1" 200 -
```

zsh project

Python: app p...

Python: migra...

Python: db pr...

Finish Setup Go Live

THUNDER CLIENT

project (Workspace)

New Request

Activity Collections Env

filter activity

DEL 127.0.0.1:5001/api/menu/721 just now

PUT 127.0.0.1:5001/api/menu/721 just now

POST 127.0.0.1:5001/api/menu just now

GET 127.0.0.1:5001/api/menu/range?start=... just now

GET 127.0.0.1:5001/api/menu/701 just now

GET 127.0.0.1:5001/api/menu just now

DELETE http://127.0.0.1:5001/api/menu/721 Send

Query Headers Auth Body Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 105 Bytes Time: 102 ms

Response Headers Cookies Results Docs

```
1 {
2   "message": "Successfully deleted item 721 from shard 1!", Copy
3   "shard_used": 1,
4   "status": "success"
5 }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
+ project /Users/tanmai/project/venv/bin/python /Users/tanmai/project/app.py
127.0.0.1 -- [18/Apr/2026 05:42:20] "PUT /api/menu/721 HTTP/1.1" 200 -
WAL: Logged DELETE - {'item_id': 721}
WAL: Committed DELETE - {'item_id': 721}
127.0.0.1 -- [18/Apr/2026 05:42:54] "DELETE /api/menu/721 HTTP/1.1" 200 -
```

zsh project

Python: app p...

Python: migra...

Python: db pr...

Finish Setup Go Live

4. SQL Shard Tables and Data Migration

Shard Tables Created

The original `MenuItem` table was divided into three shard tables:

- `shard_0_menuitem`
- `shard_1_menuitem`
- `shard_2_menuitem`

Each shard uses the **same schema** as the original table to maintain consistency.

Data Distribution Logic

Data was distributed using:

`item_id % 3`

- Result = 0 → `shard_0`
- Result = 1 → `shard_1`
- Result = 2 → `shard_2`

Data Migration Process

Migration was performed using:

```
INSERT INTO shard_X_menuitem  
SELECT * FROM menuitem WHERE item_id % 3 = X;
```

Validation Checks

After migration:

- Total row count across shards = original table count
- No duplicate `item_id` across shards
- Balanced distribution observed

Final distribution:

Shard	Count
Shard 0	7
Shard 1	5
Shard 2	7

This confirms correct and efficient data partitioning.

5. Sharding Approach and Isolation

The system implements **logical sharding using multiple tables within a single database instance**, rather than physical sharding using Docker containers or multiple database servers.

Sharding Approach

- The original `MenuItem` table is horizontally partitioned into three shard tables:
 - `shard_0_menuitem`
 - `shard_1_menuitem`
 - `shard_2_menuitem`
- Each shard stores a **disjoint subset of data** based on the hash function:

$\text{shard_id} = \text{item_id} \% 3$

- This ensures that every record exists in **exactly one shard**, with no duplication.

Unlike distributed sharding (multiple databases or Docker instances), this approach keeps all shards within the same database, simplifying deployment while still demonstrating core sharding principles.

Isolation Mechanism

Isolation between shards is achieved at the **application and schema level**:

- **Table-level isolation:**
Each shard is stored in a separate table, ensuring physical separation of data within the database.

- **Routing-based isolation:**

The application uses a shard router (`get_shard_connection(item_id)`) to ensure that:

- Queries access only the relevant shard
- No unnecessary cross-shard queries occur for single-key operations

- **No cross-shard interference:**

Since each `item_id` maps deterministically to one shard, operations such as insert, update, and delete affect only that shard.

- **Controlled multi-shard access:**

Queries that require data from multiple shards (e.g., full table scan or range queries) explicitly fetch data from all shards and merge results at the application layer.

This approach achieves effective shard isolation through:

- Table-level data separation
- Deterministic routing logic
- Controlled application-level access

Although all shards reside within a single database instance, the system successfully models the behavior of a distributed sharding system while maintaining simplicity.

```
PS C:\Users\Anusha Modayalalasa\Downloads\project_2\project> python app.py
WAL RECOVERY: Replaying BOOK → {'member_id': 4, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 4, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 5, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 6, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 4, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 2, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 3, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 9, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 2, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 7, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 1, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 4, 'total': 230}
WAL RECOVERY: Replaying BOOK → {'member_id': 5, 'total': 230}
Recovered 56 operation(s) from WAL
B+ Tree fully synced from all shards
* Debugger is active!
* Debugger PIN: 117-899-162
127.0.0.1 - - [17/Apr/2026 03:40:40] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 03:40:40] "GET /favicon.ico HTTP/1.1" 404 -
```

6. Scalability and Trade-offs Analysis

6.1 Horizontal vs Vertical Scaling

- Vertical scaling → limited by hardware
- Horizontal scaling (used here) → scalable by adding shards

Advantages of sharding:

- Load distributed across shards
- Increased system capacity
- Parallel query execution

Hence, horizontal scaling provides better long-term scalability.

6.2 Consistency

- Strong consistency for single-shard queries
- Eventual consistency for multi-shard queries

Reason:

- Single-key queries → exact shard access
- Multi-shard queries → require merging results

6.3 Availability

- Failure of one shard affects only part of data
- Other shards remain functional

Result:

- Partial availability
- Improved fault isolation

6.4 Partition Tolerance

- Each shard works independently
- System continues functioning during failures

Behavior:

- Single-shard queries → may fail
- Multi-shard queries → return partial results

7. Observations and Limitations

Observations

- Hash-based partitioning produced near-uniform distribution
- Query routing is efficient with $O(1)$ shard lookup
- System scales easily by adding shards
- No duplication or data loss observed

Limitations

- Range queries require querying all shards (higher cost)
- No replication → risk of data loss on shard failure
- Slight imbalance due to small dataset size
- Increased complexity compared to single database

Conclusion

The implemented sharding system successfully demonstrates:

- Efficient shard key selection
- Balanced data distribution
- Optimized query routing
- Improved scalability over traditional systems

The design meets all assignment requirements and provides a solid foundation for scalable database systems.